DATA CORE

Programme Omega Logistics – Rapport professionnel

Certification Data Engineer – Blocs 1 à 4 (C1-C21)

Jalon transverse M4 – Restitution BI (C16bis, C20bis)

Alexandre Laugier
21 octobre 2026

Table des matières

1 Introduction **3**

2 Bloc 1 — Cadrage (C1-C7) **4**
2.1 C1 — Analyser l'expression d'un besoin 4
2.2 C2 — Cartographier les données disponibles 4
2.3 C3 — Concevoir un cadre technique d'exploitation 6
2.4 C4 — Réaliser une veille technique et réglementaire 6
2.5 C5 — Planifier la réalisation d'un projet data 6
2.6 C6 — Superviser la réalisation d'un projet data 6
2.7 Versionnement et workflow de développement 7
2.8 C7 — Communiquer tout au long du projet 7
2.9 Écart constaté et traité avant clôture 7

3 Bloc 2 — Collecte & Stockage (C8-C12) **9**
3.1 Séquencement retenu 9
3.2 C8 — Automatiser l'extraction de données 9
3.3 C9 — Développer des requêtes SQL d'extraction 10
3.4 C10 — Développer des règles d'agrégation de données 10
3.5 C11 — Créer une base de données 10
3.6 C12 — Partager le jeu de données 12
3.7 Point ouvert transmis au bloc suivant 12

4 Bloc 3 — Entrepôt OMEGA BI (C13-C17) **13**
4.1 Séquencement retenu 13
4.2 C13 — Modéliser un entrepôt de données 13
4.3 C14 — Créer un entrepôt de données 14
4.4 C15 — Développer des pipelines de données 16
4.5 C16 — Assurer la gestion opérationnelle d'un environnement de données 16
4.6 C17 — Mettre en œuvre une variation de dimension 16

5 Bloc 4 — Data Lake OMEGA LAKE (C18-C21) **17**
5.1 C18 — Concevoir l'architecture du *data lake* 17
5.2 C19 — Intégrer une infrastructure de données à partir de composants 18
5.3 C20 — Développer un catalogue de données 19
5.4 C21 — Garantir la gouvernance des données 19

6 Jalon M4 — Restitution BI (Grafana) **20**
6.1 Choix de l'outil 20
6.2 C16bis — Restitution de l'entrepôt OMEGA BI 20
6.3 C20bis — Monitorage du *data lake* 21
6.4 Infrastructure : lancer et accéder aux services 21
6.5 Rejouer le remplissage des données et les pipelines 22

6.5.1 Transactions et rollback 23
6.5.2 Visualiser les pipelines 23
6.6 Portée budgétaire 24

7 Synthèse transverse **25**

8 Conclusion **26**

A Glossaire métier **27**

B Matrice compétences **30**

1. Introduction

2. Bloc 1 — Cadrage (C1-C7)

Le Bloc 1 pose le cadrage complet du programme DATA CORE pour Omega Logistics : analyse du besoin, cartographie des données existantes, architecture cible, veille, planification, supervision et communication. Clos le 25/08/2026 (milestone M0), les 7 compétences ont chacune un livrable produit et vérifié – détail exhaustif dans `docs/comptes_rendus/M0.md` et les documents cités ci-dessous.

2.1 C1 — Analyser l'expression d'un besoin

L'étude de faisabilité (`docs/architecture/etude_faisabilite.md`) s'appuie sur des entretiens avec les parties prenantes du programme (commanditaire opérationnel, sponsor, responsables d'entrepôt), une analyse d'opportunité et de faisabilité, une priorisation RICE des blocs de compétences, et des objectifs formulés selon la méthode SMART. Cette analyse RICE a directement arbitré le séquencement retenu pour la suite du programme (collecte avant entrepôt, entrepôt avant data lake).

2.2 C2 — Cartographier les données disponibles

La topographie des données (`docs/architecture/topographie_donnees.md`) couvre le glossaire métier et les modèles réels des sept sources identifiées – FluxPro (WMS), TransFlow (TMS), le portail transporteur, les fichiers clients hétérogènes, l'historique volumineux et les deux flux IoT anticipant le bloc 4 – en les explorant directement dans `data/raw/` et `api-mock/`, plutôt qu'en se fiant à une description théorique des sources.

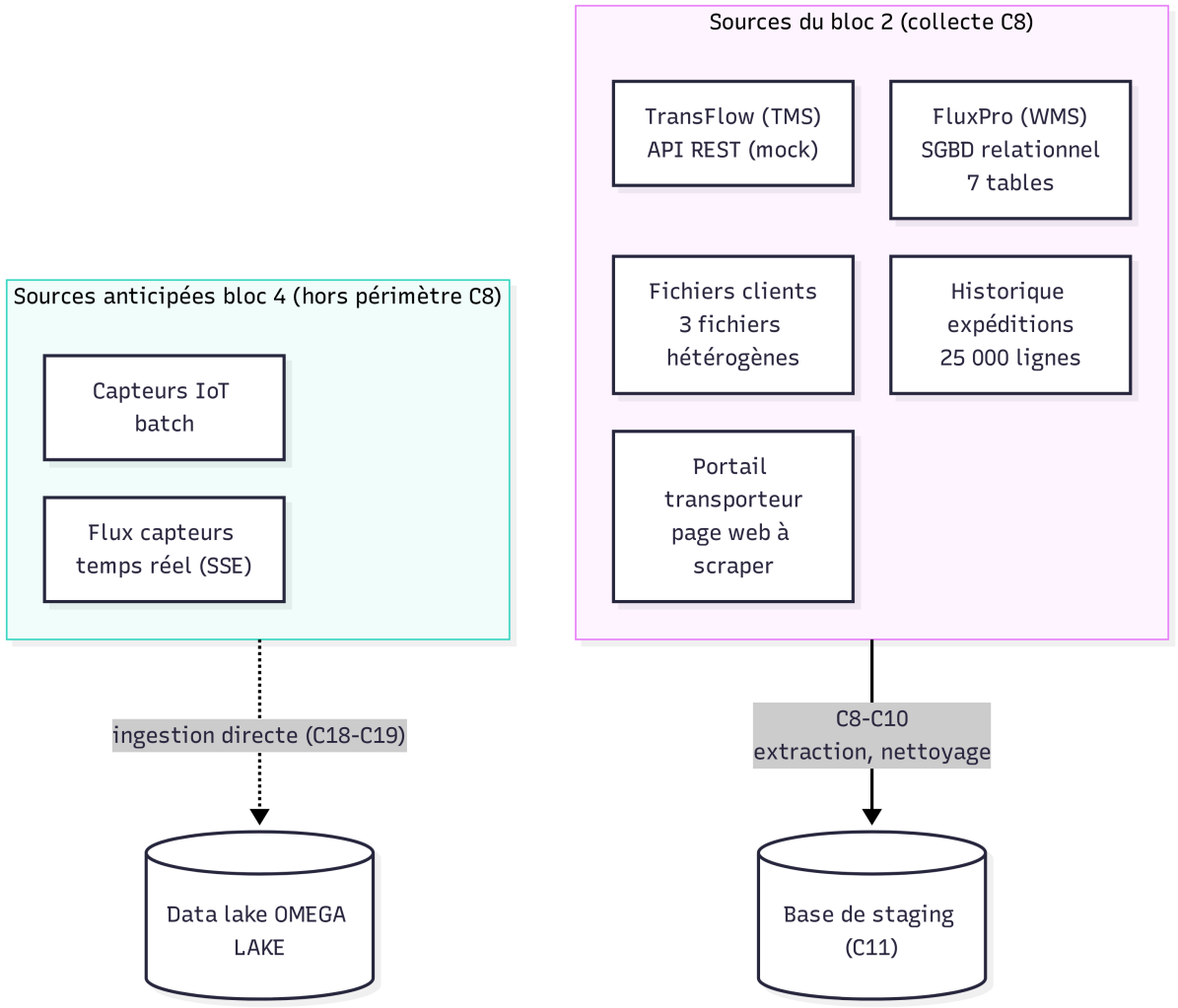


FIGURE 2.1 – Cartographie des sources de données et de leur destination (`docs/architecture/topographie_donnees.md`)

2.3 C3 — Concevoir un cadre technique d'exploitation

L'étude technique d'architecture (`docs/architecture/architecture_cible.md`) établit la cible AS IS/TO BE, une matrice des flux, un registre RGPD préliminaire par brique technique, et la stratégie d'éco-conception (RGESN) et d'accessibilité (RGAA) du programme. C'est à ce stade qu'est actée la décision structurante de retenir MinIO plutôt que Microsoft Fabric pour le futur data lake : contrainte de fonctionnement local sans compte cloud, coût, reproductibilité pour un évaluateur tiers – une décision confirmée et revérifiée par la veille technique du même jour.

2.4 C4 — Réaliser une veille technique et réglementaire

Un journal de veille hebdomadaire sur trois semaines (`docs/architecture/veille_technique_reglementaire.md`) couvre deux axes, technique (DataOps/ETL) et réglementaire (RGPD/RGAA) – chaque entrée est reliée à une décision réelle du programme, pas tenue comme un exercice isolé.

2.5 C5 — Planifier la réalisation d'un projet data

La feuille de route (`docs/architecture/feuille_de_route.md`) découpe le programme en 4 phases correspondant aux 4 blocs de compétences, avec composition d'équipe, budget prévisionnel (180 000 €) et diagrammes de Gantt/PERT – maintenus en Mermaid, c'est-à-dire en texte versionné dans le dépôt, plutôt que dans un outil graphique externe. GitHub Projects est retenu comme outil de suivi, directement intégré à la convention de branches du projet.

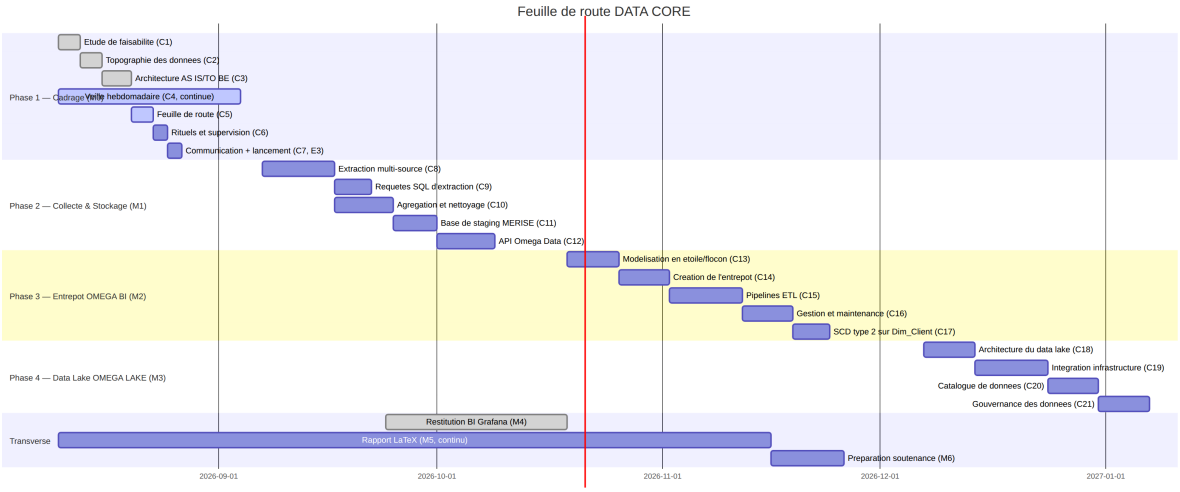


FIGURE 2.2 – Diagramme de Gantt du programme (export du Mermaid source, `docs/architecture/feuille_de_route.md`)

2.6 C6 — Superviser la réalisation d'un projet data

La supervision du programme (`docs/architecture/supervision_projet.md`) définit les rituels d'équipe (point hebdomadaire, comité de suivi mensuel) et le tableau de suivi budgétaire, actualisé à chaque jalon. Sur l'ensemble du programme (M0 à M4), le budget du périmètre initial (180 000 €) a été consommé intégralement, sans dépassement – voir le chapitre 7 pour la vue consolidée.

2.7 Versionnement et workflow de développement

Le code et la documentation sont versionnés dans un unique dépôt GitHub, selon une convention actée dès M0 et appliquée sans exception sur les 5 jalons suivants (`docs/plan_de_developpement.md`) : branche `main` protégée (releases uniquement), branche `dev` d'intégration continue, et une branche `feat-<numéro>-<slug>` par issue GitHub, créée depuis `dev`. Toute Pull Request d'une branche de travail cible `dev`, jamais `main` directement ; `dev` ne rejoint `main` que lors d'une release de fin de jalon, validée par une PR dédiée.

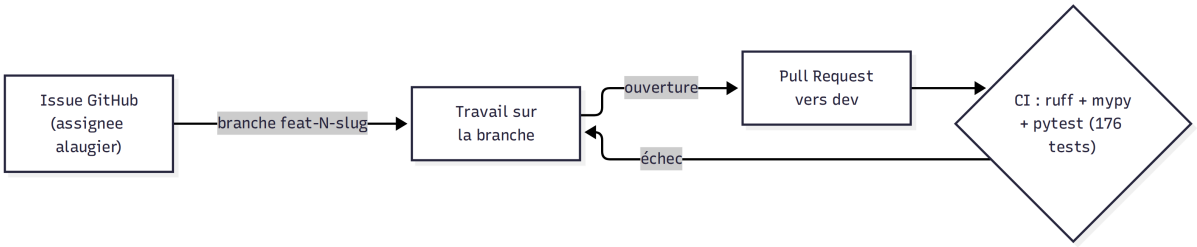


FIGURE 2.3 – Cycle de vie d'une tâche (1/2) – développement, jusqu'à la CI (`docs/architecture/supervision_projet.md`)

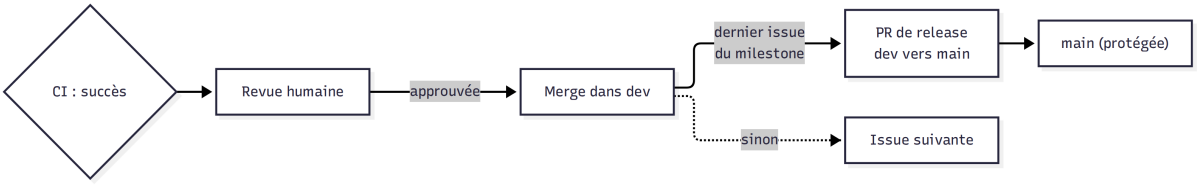


FIGURE 2.4 – Cycle de vie d'une tâche (2/2) – revue et intégration, jusqu'à la release (`docs/architecture/supervision_projet.md`)

Chaque Pull Request déclenche une intégration continue (`.github/workflows/ci.yml`) en trois étapes séquentielles – `ruff` (lint), puis `mypy` (typage statique), puis `pytest` sur `tests/unit` et `tests/integration` (176 tests au 21/10/2026) – les tests ne s'exécutent que si lint et typage passent déjà, pour éviter un rapport d'échec bruité par des erreurs de surface. Le suivi visuel du travail (GitHub Projects, 6 statuts : *Backlog* → *À faire (sprint)* → *En cours* → *En revue (PR)* → *Bloqué* → *Terminé*) suit ce même cycle, mis à jour manuellement à chaque étape – un mécanisme d'automatisation par *linked PR* a été essayé puis abandonné le 25/08/2026, trop rigide pour les PR de release qui ferment plusieurs issues à la fois. À la clôture du dernier issue d'un jalon, un compte rendu (`docs/comptes_rendus/`) et la release vers `main` sont produits dans la même PR, avant de fermer le milestone GitHub correspondant.

2.8 C7 — Communiquer tout au long du projet

Le plan de communication et les supports de lancement (`docs/architecture/communication_lancement.md`) couvrent la communication aux parties prenantes à chaque étape clé du programme, avec une attention portée à l'accessibilité des documents partagés (RGAA).

2.9 Écart constaté et traité avant clôture

Le suivi GitHub initial du Bloc 1 (issues #1 à #8) ne couvrait que 4 des 7 compétences attendues – gap identifié à la première rédaction du compte rendu de milestone, comblé par

l'ouverture de 3 issues supplémentaires (C4, C6, C7) avant de considérer M0 clos. Ce réflexe de vérification — confronter le suivi du dépôt au périmètre réellement exigé, plutôt que de supposer la couverture complète — est resté une pratique constante sur l'ensemble du programme (voir notamment le réexamen du périmètre de C20 au chapitre 5).

3. Bloc 2 — Collecte & Stockage (C8-C12)

Le Bloc 2 industrialise la collecte multi-source, nettoie et consolide les données dans une base de travail, puis les expose via une API. Clos le 27/08/2026 (milestone M1), les 5 compétences ont chacune un livrable vérifié – détail exhaustif dans `docs/comptes_rendus/M1.md`.

3.1 Séquencement retenu

L'ordre C8 → C10 → C9 → C11 → C12 (`docs/architecture/sequencement_bloc2.md`), documenté et justifié avant tout développement plutôt que décidé au fil de l'eau, diffère de l'ordre naturel de lecture du cahier des charges (qui aurait suggéré de modéliser avant d'agréger). L'expérience a validé ce choix : le grain réel des doublons dans les fichiers clients n'a été découvert qu'en les traitant concrètement, pas en les modélisant par anticipation.

3.2 C8 — Automatiser l'extraction de données

Cinq connecteurs couvrent l'ensemble des sources identifiées en C2 : API REST (TransFlow), scraping du portail transporteur, requêtes SQL directes (FluxPro), fichiers clients hétérogènes, et l'historique volumineux – testés de bout en bout (31 tests).

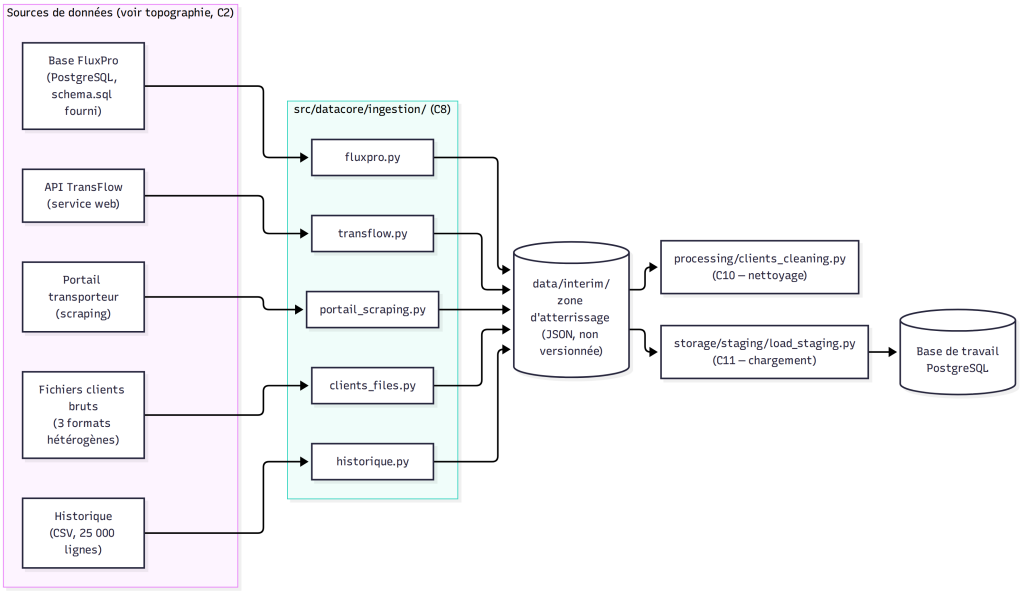


FIGURE 3.1 – Diagramme de séquence de l'extraction multi-source (`docs/architecture/diagrammes_techniques.md`)

3.3 C9 — Développer des requêtes SQL d'extraction

Cinq requêtes documentées avec leurs résultats réels (`docs/architecture/requetes_sql_extraction.md`). L'historique est chargé dans la base de staging PostgreSQL existante plutôt que dans un outil « big data » séparé : au volume réel du jeu de données (25 000 lignes), un outil dédié aurait été disproportionné — choix justifié par la sobriété numérique (RGESN) plutôt que par principe.

3.4 C10 — Développer des règles d'agrégation de données

Le nettoyage et le dédoublonnage des fichiers clients sont menés au bon grain (commande, produit) : une première mesure au grain « commande » seul donnait un taux de 55 à 68 % de doublons, chiffre trompeur qui mesurait en réalité le nombre moyen de produits par commande (une commande multi-produits génère plusieurs lignes légitimes). Au bon grain (commande, produit), le taux réel mesuré tombe à 13 à 17 % selon le client :

Listing 3.1 – Mesure du taux de doublons au grain (commande, produit) (`notebooks/exploration_fichiers_clients.ipynb`)

```
for name, (path, sep, key, prod_key) in FILES.items():
    rows = rows_by_client[name]
    grain_counts = Counter((r[key], r[prod_key]) for r in rows)
    true_dups = {k: v for k, v in grain_counts.items() if v > 1}
    print(f"{name}: {len(rows)} lignes, {len(grain_counts)} couples "
          f"(commande, produit) uniques, {len(true_dups)} vrais "
          f"doublons ({len(true_dups) / len(grain_counts):.1%})")
```

`key`/`prod_key` désignent respectivement la colonne identifiant de commande et la colonne produit (SKU) de chaque fichier client – `grain_counts` compte les occurrences de chaque couple (commande, produit) ; un couple vu plus d'une fois (`true_dups`) est un vrai doublon à ce grain. Ce chiffre, corrigé après une première mesure erronée, est celui retenu dans `docs/architecture/topographie_donnees.md` et implémenté dans `deduplicate()` (`src/datacore/processing/clients_cleaning.py`).

3.5 C11 — Créer une base de données

La base de travail est modélisée selon la méthode MERISE (`docs/architecture/modelisation_merise.md`), versionnée via Alembic.

L'analyse des formes normales a été menée empiriquement — en interrogeant réellement les données — plutôt que théoriquement, et a révélé deux violations réelles, corrigées après vérification chiffrée : `livraisons.statut` (3NF, remplacé par une vue SQL) et `commandes_clients` (2NF, décomposée en en-tête/lignes).

La clé métier de rapprochement entre livraisons et expéditions FluxPro est `tracking_number`, portée sans contrainte de clé étrangère formelle (les deux tables appartiennent à des systèmes sources distincts) :

Listing 3.2 – Clé de rapprochement `tracking_number` (`src/datacore/storage/staging/models.py`)

```
# tracking_number : cle metier de rapprochement avec
# expeditions.tracking_number (FluxPro) -- pas de contrainte FK
# (systemes sources distincts), rapprochement fait a la lecture.
Column("tracking_number", String(20), nullable=False),
```

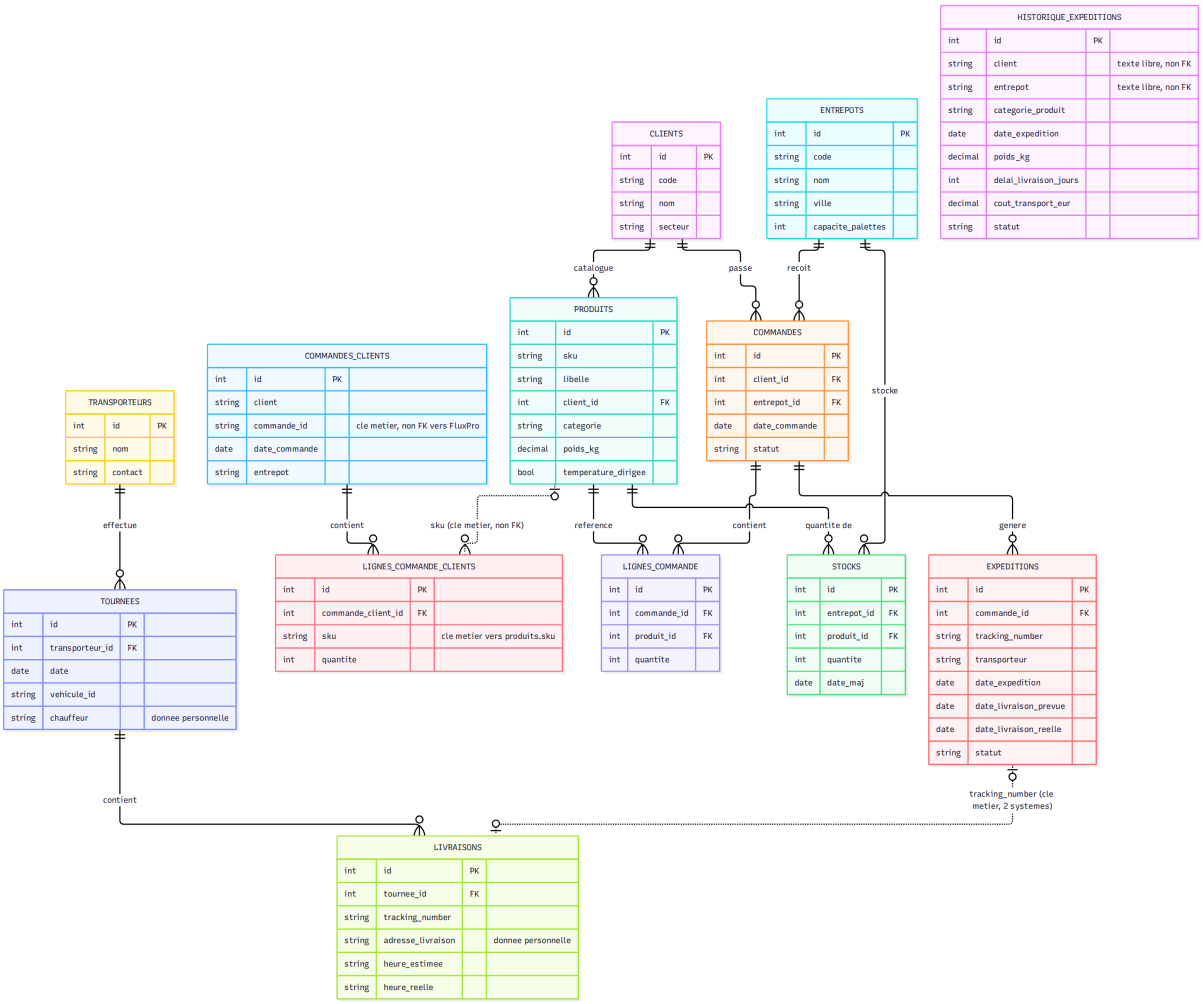


FIGURE 3.2 – Modèle conceptuel de données de la base de travail (`docs/architecture/modelisation_merise.md`)

3.6 C12 — Partager le jeu de données

L'API Omega Data (FastAPI, choisie pour la génération automatique de la spécification OpenAPI exigée par le livrable) expose le jeu de données consolidé, avec authentification par clé et autorisation par rôle – les référents clients externes sont systématiquement restreints à leur propre client, règle reprise directement du registre RGPD établi en C11. Le comportement le plus important n'est pas visible dans un diagramme d'architecture statique : un référent client n'est pas seulement *rejeté* s'il demande les données d'un autre client, il est *silencieusement ramené* à son propre périmètre – vérifié en conditions réelles (`notebooks/demo_omega_data_api.ipynb`).

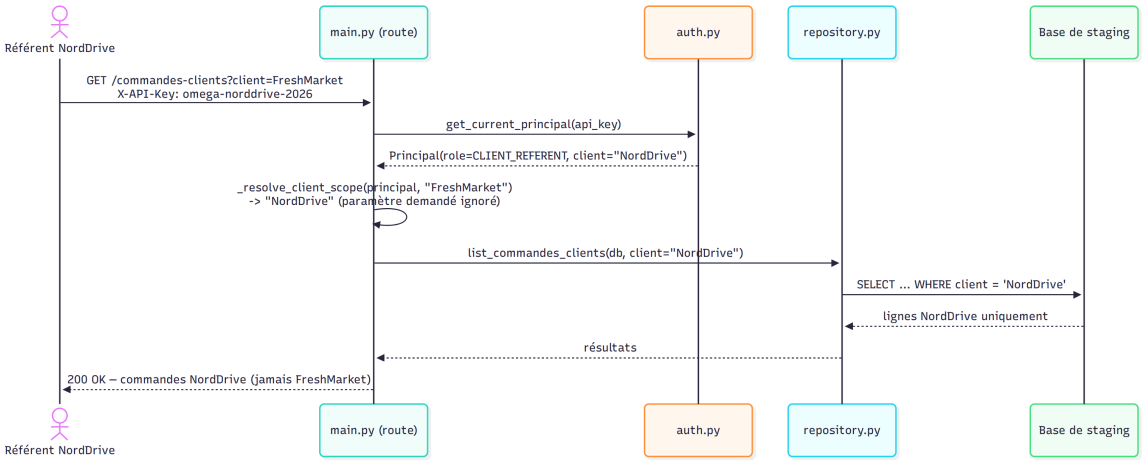


FIGURE 3.3 – Séquence d'autorisation de l'API Omega Data (`docs/architecture/diagrammes_techniques.md`)

3.7 Point ouvert transmis au bloc suivant

Le rapprochement entre `clients.nom` (FluxPro) et le champ texte libre `historique_expeditions.client` reste non garanti par une contrainte formelle à ce stade — signalé comme point à traiter explicitement lors de la modélisation de l'entrepôt de données (chapitre 4).

4. Bloc 3 — Entrepôt OMEGA BI (C13-C17)

Le Bloc 3 construit et maintient l'entrepôt décisionnel OMEGA BI, qui répond au besoin d'analyse de la performance logistique exprimé par la direction. Clos le 21/09/2026 (milestone M2), les 5 compétences ont chacune un livrable vérifié – détail exhaustif dans `docs/comptes_rendus/M2.md`.

4.1 Séquencement retenu

L'ordre C13 → C14 → C15 → C17 → C16 (`docs/architecture/sequencement_bloc3.md`) inverse volontairement C16/C17 par rapport au Gantt initial : la procédure de purge RGPD de C16 dépend du schéma historisé que C17 doit d'abord créer.

4.2 C13 — Modéliser un entrepôt de données

Approche bottom-up par datamarts (Kimball) : 2 datamarts, 3 faits, 6 dimensions conformées, matrice de bus (`docs/architecture/modelisation_omega_bi.md`). Le seul schéma en flocon retenu (`Dim_Produit` → `Dim_Categorie`) est justifié par une contrainte réelle des sources, pas par principe de modélisation.

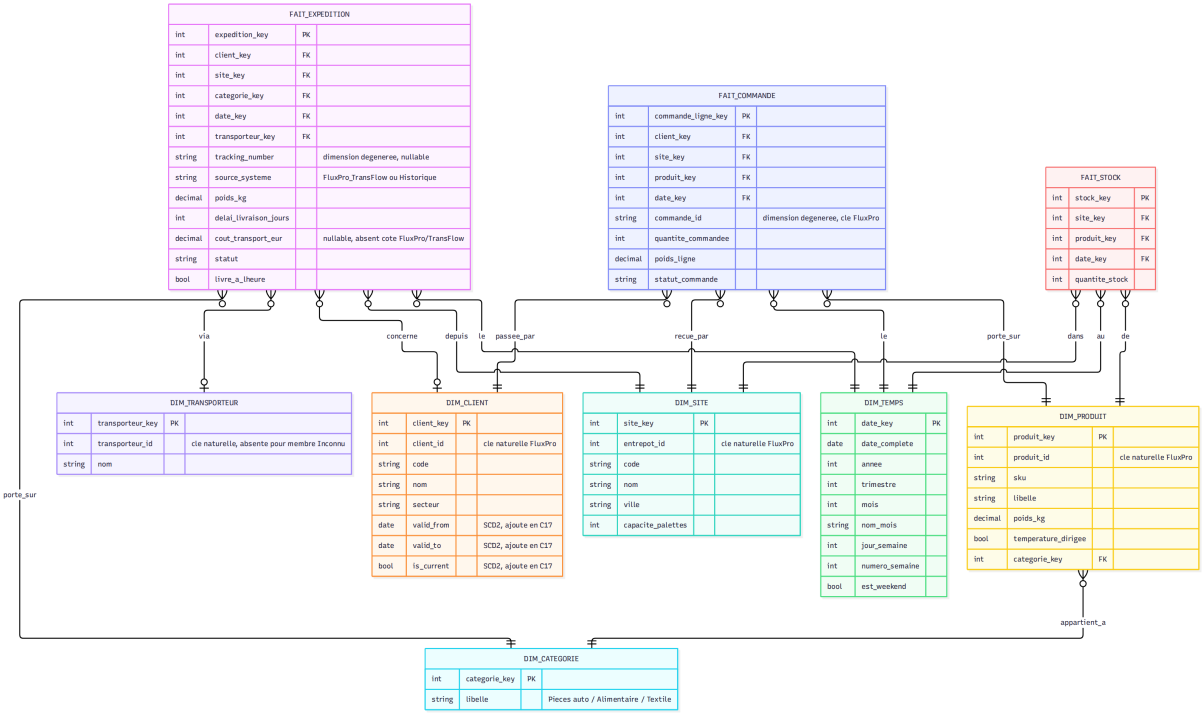


FIGURE 4.1 – Modèle conceptuel de données de l'entrepôt OMEGA BI (`docs/architecture/modelisation_omega_bi.md`)

Pourquoi pas de `Fait_Livraison` séparée : `livraisons` (staging, TransFlow) et `expeditions` (FluxPro) décrivent le même événement métier vu par deux systèmes sources, rapprochés par `tracking_number` (chapitre 3 §C11). La cardinalité réelle a été vérifiée avant de modéliser, pas supposée : 100 % des 1100 expéditions FluxPro ont exactement une livraison TransFlow correspondante (relation 1 :1 confirmée empiriquement). Scinder deux sources en cardinalité 1 :1 vérifiée en deux tables de faits aurait imposé une jointure fait-à-fait à chaque requête – anti-pattern Kimball qu'une dimension conformée évite normalement. Les mesures de `livraisons` (délai, statut) sont donc absorbées directement dans `Fait_Expedition`, au même grain.

4.3 C14 — Créer un entrepôt de données

L'entrepôt est créé dans une base PostgreSQL distincte de la base de staging, mais sur la même instance (sobriété RGESN) – 3 schémas physiques créés à ce stade (`dimensions`, `exploitation`, `commercial`), versionnés via un second environnement Alembic dédié. Le cycle `upgrade`/`downgrade` est vérifié contre une base réelle, pas seulement contre une base vide (un bug latent d'Alembic, `include_schemas=True` manquant, n'a d'ailleurs été révélé qu'à la seconde migration). Deux schémas supplémentaires rejoignent la même base par la suite, sans modifier les trois premiers : `gouvernance` (journalisation des opérations, C16) et `monitoring` (métriques du data lake, jalon M4/C20bis, chapitre 6).

Seules les tables de faits référencent les dimensions par clé étrangère inter-schéma ; `gouvernance` et `monitoring` sont des journaux techniques autonomes, jamais joints aux dimensions. `bi_reader` (lecture seule) a accès à tous les schémas sauf `gouvernance`, délibérément exclu (préoccupation d'exploitation interne, pas de restitution BI).

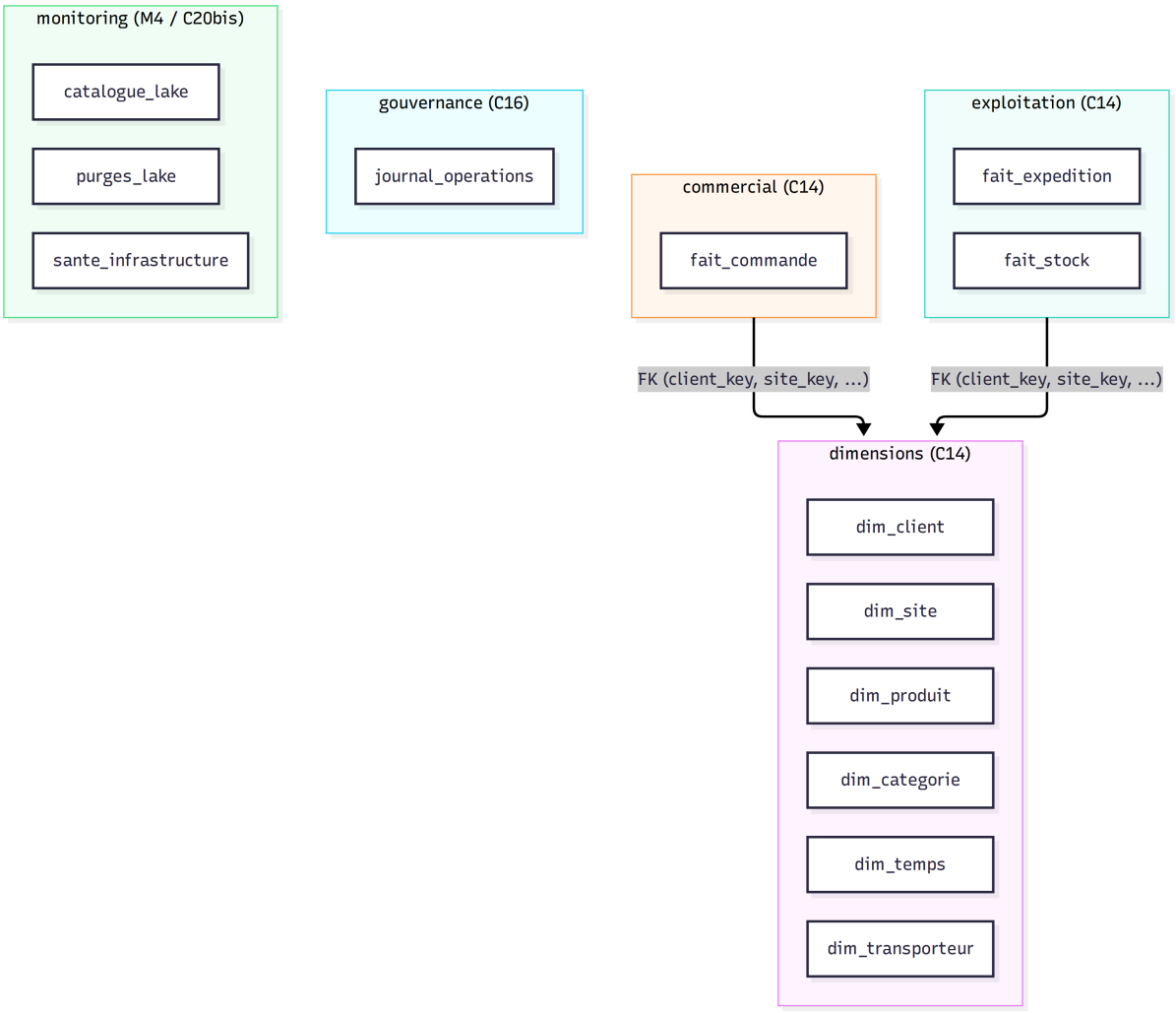


FIGURE 4.2 – Les 5 schémas PostgreSQL de `datacore_omega_bi` et leurs liens (`docs/architecture/creation_entrepot_omega_bi.md`)

4.4 C15 — Développer des pipelines de données

Le pipeline ETL staging → entrepôt applique des contrôles qualité réels (mise en quarantaine des lignes non rapprochées de l'historique, pas un rejet silencieux), et sa cohérence est vérifiée en comparant les indicateurs calculés par l'ETL à ceux calculés indépendamment par l'API C12 – deux chemins de calcul distincts produisant le même résultat.

4.5 C16 — Assurer la gestion opérationnelle d'un environnement de données

La gestion opérationnelle (`docs/architecture/gestion_operationnelle_omega_bi.md`) couvre des sauvegardes planifiées (partielle/complète, justifiées par la reconstructibilité de 7 des 8 tables depuis le staging), une journalisation réelle des opérations dans le nouveau schéma `gouvernance` (succès et échec, sur une connexion dédiée pour survivre à un `ROLLBACK`), et un registre RGPD appuyé sur un audit automatisé répétable ne trouvant aucune donnée personnelle dans l'entrepôt.

Ce périmètre a été étendu après la clôture initiale du bloc : la restitution par vues SQL et notebook s'est révélée insuffisante comme preuve de restitution, et renforcée dans un outil de BI dédié – traité comme son propre jalon transverse (chapitre 6).

4.6 C17 — Mettre en œuvre une variation de dimension

`Dim_Client` est historisée en SCD2 (type 2 de Kimball, voir aussi SCD1) : un changement de `nom` ou `secteur` ferme la version courante et en ouvre une nouvelle, plutôt que d'écraser la valeur précédente.

Listing 4.1 – Détection du changement et historisation SCD2 (`src/datacore/storage/warehouse/load_warehouse.py`)

```
if current is not None and current[1] == r["nom"] and current[2] == r["
    secteur"]:
    mapping[r["id"]] = current[0]
    continue

if current is not None:
    cur.execute(
        """
        UPDATE dimensions.dim_client
        SET valid_to = %(today)s, is_current = false
        WHERE client_key = %(client_key)s
        """,
        {"today": today, "client_key": current[0]},
    )
# ... puis INSERT d'une nouvelle ligne avec une nouvelle client_key.
```

`r` est la ligne source lue depuis le staging (dict avec `nom`/`secteur`/`id` FluxPro) ; `current` est la version actuellement courante de ce client dans l'entrepôt si elle existe déjà (`None` sinon), sous forme de tuple `(client_key, nom, secteur)` ; `mapping` associe l'identifiant FluxPro à la `client_key` à utiliser pour charger les faits qui le référencent, qu'une nouvelle version ait été créée ou non. Le mécanisme est vérifié par un changement réel simulé : l'ancienne version reste close et interrogeable, la nouvelle est créée, et les faits déjà chargés continuent de référencer la version qui était courante au moment de leur chargement. Écart assumé avec la description initiale de l'issue (qui demandait l'historisation d'attributs absents du jeu de données pédagogique, adresse et contrat) : `nom` et `secteur` ont été retenus à la place, après décision prise avant implémentation plutôt que devinée.

5. Bloc 4 — Data Lake OMEGA LAKE (C18-C21)

Le Bloc 4 conçoit et opère le data lake OMEGA LAKE pour absorber la donnée massive IoT de la flotte et des entrepôts. Clos le 23/09/2026 (milestone M3), les 4 compétences ont chacune un livrable vérifié – détail exhaustif dans `docs/comptes_rendus/M3.md`.

5.1 C18 — Concevoir l'architecture du *data lake*

La différence entre base de staging, entrepôt et data lake n'est pas le volume ni le degré de nettoyage des données, mais le moment où le schéma est décidé : *schema-on-write* pour les deux premiers, *schema-on-read* pour le lake (`docs/architecture/architecture_omega_lake.md`). Les flux IoT transitent en flux séparé (jamais par la base de staging relationnelle), mais la zone `curated/` est conçue pour être jointe à la volée aux dimensions conformées de l'entrepôt via des clés naturelles réelles, vérifiées dans les données plutôt que supposées.

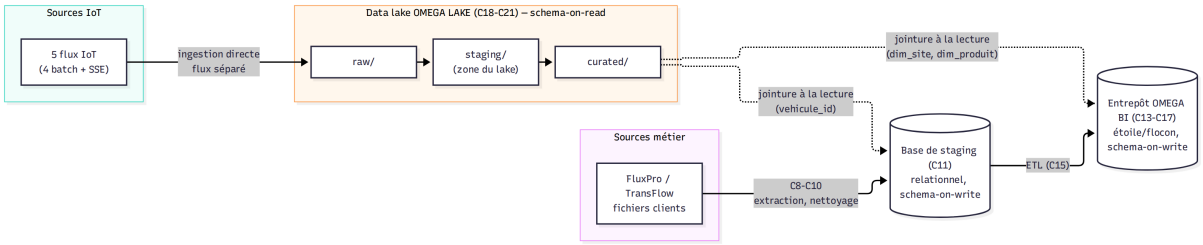


FIGURE 5.1 – Vue d'ensemble : base de staging, entrepôt OMEGA BI et data lake OMEGA LAKE (`docs/architecture/architecture_omega_lake.md`)

La figure ci-dessus situe le data lake par rapport aux deux autres briques de stockage déjà couvertes (chapitres 3 et 4) : les flux IoT y entrent en ingestion directe (flux séparé de la base de staging), et c'est la zone `curated/` qui se joint à la lecture aux dimensions conformées de l'entrepôt – jamais l'inverse, et jamais de duplication entre les trois briques.

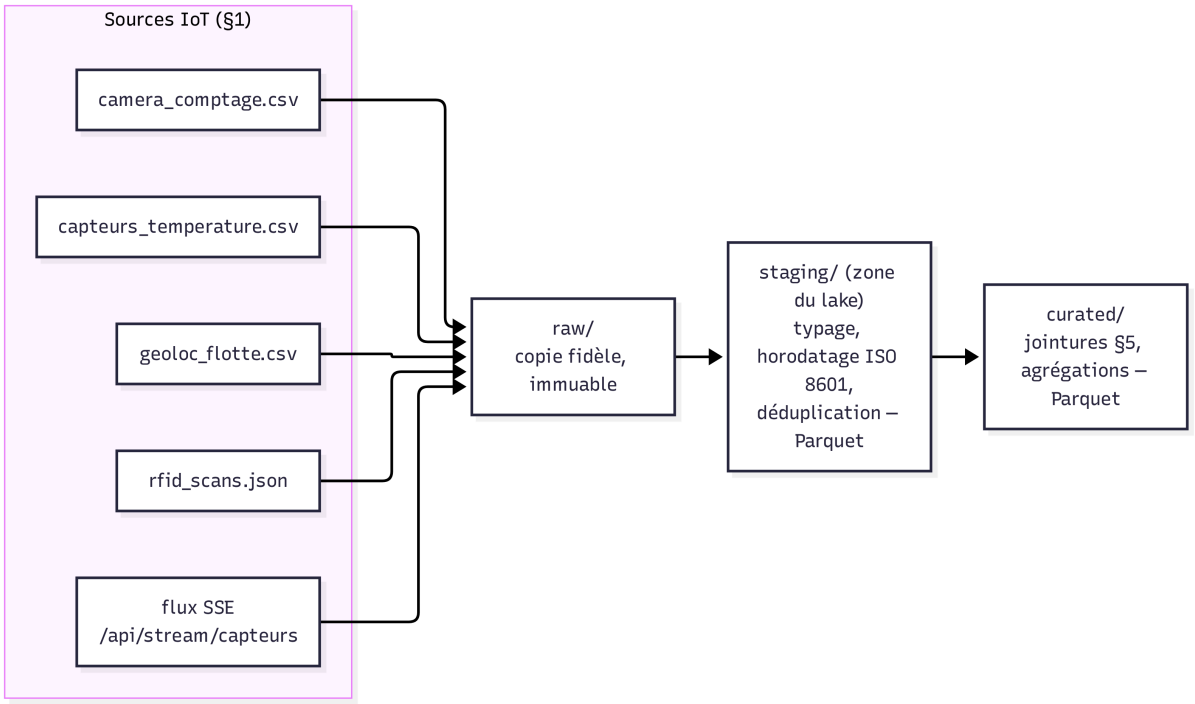


FIGURE 5.2 – Zones `raw`/`staging`/`curated` du data lake OMEGA LAKE (`docs/architecture/architecture_omega_lake.md`)

5.2 C19 — Intégrer une infrastructure de données à partir de composants

MinIO est déployé comme stockage objet compatible S3 (`docs/architecture/integration_infrastructure_omega_lake.md`). Le mécanisme de jointure — DuckDB, choisi après vérification que le projet n'utilise `pandas` nulle part — est tranché explicitement avant tout code de pipeline, et vérifié contre MinIO/PostgreSQL réels, pas supposé équivalent à un test local :

Listing 5.1 – Configuration DuckDB/`httpfs` pour MinIO (`src/datacore/storage/lake/transform.py`)

```
SET s3_endpoint='{OMEGA_LAKE_S3_ENDPOINT}';
SET s3_access_key_id='{MINIO_ROOT_USER}';
SET s3_secret_access_key='{MINIO_ROOT_PASSWORD}';
SET s3_use_ssl=false;
SET s3_url_style='path';
```

L'ingestion de `raw/` passe par `boto3` (copie fidèle, octet pour octet, vérifiée par somme de contrôle SHA-256) et non par DuckDB. La raison technique précise : lire un CSV/JSON avec DuckDB puis le réécrire peut reformater silencieusement le contenu sans changer l'information portée (quotage des champs CSV différent, ordre des clés JSON différent) – un comportement acceptable pour une jointure analytique en lecture (C18), mais incompatible avec la garantie de `raw/` : une copie strictement fidèle à l'octet près, vérifiable par somme de contrôle. `boto3` transfère des octets sans les interpréter ; c'est le bon outil pour ce rôle précis, DuckDB restant réservé aux étapes de lecture structurée (jointures, transformations `staging/`/`curated/`).

Le consommateur du flux SSE, fenêtré par intervalle de 5 minutes plutôt que par jour, a été vérifié par une coupure brutale réelle (`kill -9` en cours de réception puis redémarrage) : aucune collision ni donnée corrompue.

5.3 C20 — Développer un catalogue de données

Le périmètre de cette compétence a été explicitement élargi avant tout code : le texte initial de l'issue ne parlait que de cataloguer « après C19 », mais à ce stade les zones `staging/`/`curated/` restaient vides – construire un contenu réel y a été ajouté. Le catalogue lui-même est généré par introspection réelle (`boto3` + `DESCRIBE` DuckDB), pas maintenu à la main, même principe que l'audit RGPD automatisé de C16 : un catalogue généré ne peut pas devenir silencieusement obsolète.

5.4 C21 — Garantir la gouvernance des données

Le registre RGPD du lake (`docs/architecture/registre_rgpd_lake.md`) identifie un risque réel de ré-identification, trouvé en concevant et non en auditant après coup : `vehicule_id` (géolocalisation) se recoupe, dans le temps, avec `tournees.chauffeur` de la base de staging. Décision actée dès C18 : aucune jointure automatisée entre les deux.

Une première analyse de pseudonymisation (« pas de protection réelle possible, vu le faible nombre de véhicules ») s'est révélée incomplète après relecture externe : vraie pour les Data Engineers (accès complet des deux côtés), fausse pour le rôle `lake_reader` (accès au lake seul). Un pseudonyme HMAC à clé secrète — un hash simple sans clé aurait été recalculable par force brute sur les 15 véhicules du jeu de données — a été implémenté comme export dédié (`curated_bi/`), sans jamais modifier `curated/` ni distribuer la clé au rôle de lecture. Le mécanisme lui-même est une fonction pure, stable pour un même identifiant mais impossible à recalculer sans la clé :

Listing 5.2 – Pseudonymisation HMAC-SHA256 de `vehicule_id` (`src/datacore/storage/lake/curated_bi.py`)

```
1 def pseudonyme(vehicule_id: str, cle: str | None = LAKE_PSEUDONYM_KEY)
      -> str:
2     cle = verifier_cle_configuree(cle)
3     return hmac.new(cle.encode(), vehicule_id.encode(), hashlib.sha256).
          hexdigest()[:16]
```

`verifier_cle_configuree()` est le garde-fou qui refuse explicitement de continuer (erreur levée, pas un avertissement silencieux) si la clé est absente, vaut une valeur publique connue, ou est manifestement trop courte pour un usage HMAC sérieux. Ce garde-fou a lui-même été renforcé après qu'une faille a été trouvée en seconde relecture externe — un repli codé en dur pour cette clé dans `config.py`, visible dans le dépôt public, aurait rendu la pseudonymisation silencieusement inopérante — puis à nouveau après une question sur sa couverture au-delà du seul cas de la valeur d'exemple. Les droits d'accès par groupe ont également été corrigés en cours de route : la politique `lake_reader` portait initialement sur le bucket entier, permettant de contourner la pseudonymisation en lisant `curated/` directement – restreinte au seul préfixe `curated_bi/`, revérifiée bout en bout avec les vraies clés `lake_reader` (lecture de `curated_bi/` réussit et renvoie des données pseudonymisées ; la même requête contre `curated/` échoue en HTTP 403). La procédure de purge RGPD est vérifiée par un test réel : une partition synthétique délibérément datée, injectée dans le lake réel, purgée avec succès et le résultat confirmé indépendamment.

6. Jalon M4 — Restitution BI (Grafana)

M4 n'est pas un bloc de compétences RNCP mais un jalon transverse, ajouté en cours de programme (24/09/2026) pour combler un écart réel identifié par rapport au référentiel, et non par choix éditorial. Le référentiel C16 exige explicitement, pour l'activité 6 (maintenance de l'entrepôt), un « *suivi de l'activité de maintenance : indicateurs de service en fonction des SLA à l'aide d'un* ***tableau de bord*** », évalué sur le critère « *le tableau de bord permet de rendre compte de l'ensemble des indicateurs de service* » (référentiel, p. 19-20) ; le référentiel C20 exige de même un monitorage qui « *permet le suivi des conditions matérielles et applicatives* » et « *génère une alerte lors d'une rupture de service* » (référentiel, p. 24). Les vues SQL et le notebook exploratoire (C16) ainsi que le catalogue par introspection (C20), bien que réels et vérifiés, ne constituaient pas un tableau de bord ni un monitorage actif au sens de ces critères – un écart détecté après la clôture de M2/M3, pas un manque relevé puis jugé optionnel. M4 corrige cet écart par un outil de BI dédié (Grafana), seule forme qui satisfait littéralement les deux critères. Clos le 19/10/2026, les 2 issues du jalon sont vérifiées en conditions réelles – détail exhaustif dans `docs/comptes_rendus/M4.md`.

6.1 Choix de l'outil

Grafana est retenu, auto-hébergé, plutôt que Power BI (nécessite le Power BI Service, cloud) ou Databricks SQL/Lakeview (nécessite un *workspace* Databricks) – cohérent avec l'infrastructure 100 % locale du reste du programme, décidé avant tout codage. Le notebook SLA existant n'est ni retiré ni réécrit : il reste la preuve exploratoire telle que livrée à la clôture du Bloc 3, Grafana devenant l'outil de restitution *de production* en complément.

6.2 C16bis — Restitution de l'entrepôt OMEGA BI

Les vues SQL déjà exposées pour C16 sont restituées dans un dashboard Grafana dédié, connecté en lecture seule via le rôle `bi_reader` déjà scopé en C14. Trois bugs réels ont été trouvés uniquement en testant depuis un vrai navigateur (champ requis manquant au provisioning de la source de données, mode d'édition de requête non forcé sur les panels, agrégation silencieuse masquant deux clients sur trois sur un panel) – tous invisibles depuis les vérifications par appel direct à l'API de Grafana, qui passaient toutes. Généralisé en leçon méthodologique : une vérification par API seule ne prouve jamais qu'un rendu côté client fonctionne pour l'utilisateur final.

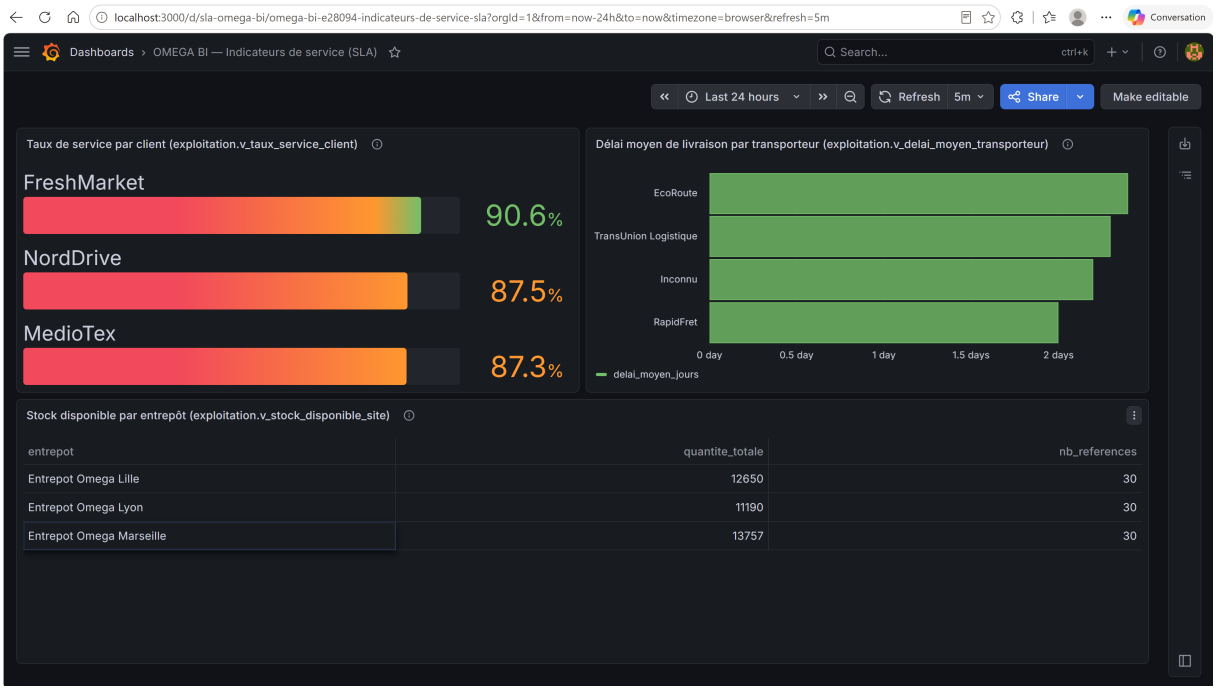


FIGURE 6.1 – Dashboard Grafana « OMEGA BI — Indicateurs de service (SLA) », confirmé fonctionnel en conditions réelles

6.3 C20bis — Monitorage du *data lake*

Une première conception du monitorage du lake, limitée au suivi de la fraîcheur des données, s'est révélée incomplète après lecture du référentiel de compétences complet (copié dans le dépôt, `docs/reference/`) : celui-ci exige explicitement le suivi des « conditions matérielles et applicatives », et qu'une alerte soit « générée lors d'une rupture de service ». Corrigé par l'ajout d'un relevé de santé de l'infrastructure (MinIO accessible, capacité utilisée) et d'une règle d'alerte Grafana réelle.

La chaîne d'alerte relie une règle Grafana, un service interne authentifié, et un relais d'e-mail réel via la bibliothèque `smtplib` vers un capteur SMTP local (MailHog) – jamais un vrai fournisseur externe, pour rester démonstrable sans identifiant réel à gérer ni envoi parasite à chaque exécution automatisée des tests. Testée de bout en bout, deux fois : une coupure réelle de MinIO déclenche un relevé en échec, l'alerte Grafana passe à l'état actif en une trentaine de secondes, et un e-mail réel est reçu dans MailHog ; la remise en service résout l'alerte. Une relecture de code a ensuite mis au jour deux trous de robustesse (un corps de requête mal formé faisait échouer le service sans réponse exploitable, un échec d'envoi d'e-mail n'était pas distingué d'une autre erreur) et l'absence d'authentification sur ce service, exposé sur l'hôte – les trois corrigés et revérifiés avant la clôture du jalon.

6.4 Infrastructure : lancer et accéder aux services

L'ensemble du programme tient dans un seul `docker-compose.yml` (`infra/docker/docker-compose.yml`), auto-hébergé, sans dépendance à un service cloud – cohérent avec la stratégie de sobriété numérique (RGESN) retenue dès C3. Depuis un terminal WSL Ubuntu (ou tout shell Linux/-macOS), à la racine du dépôt :

Listing 6.1 – Lancement de l'ensemble des services (tous blocs confondus)

```
docker compose -f infra/docker/docker-compose.yml --env-file .env up -d
```

`-env-file .env` est requis : les mots de passe (`BI_READER_PASSWORD`, `GRAFANA_ADMIN_PASSWORD`, `ALERTING_WEBHOOK_SECRET`, `LAKE_PSEUDONYM_KEY`) ne sont jamais codés en dur – un oubli de ce paramètre a d'ailleurs été le seul écart trouvé en relecture de la PR de release de M4.

Service	Port	Accès
Base PostgreSQL (staging + entrepôt)	5432	Pas d'interface web – `psql` ou client SQL
API Omega Data (C12)	8000	`http://localhost:8000/docs` (spécification OpenAPI interactive)
MinIO (console web)	9001	`http://localhost:9001`
Grafana	3000	`http://localhost:3000`
MailHog (boîte de réception des alertes)	8025	`http://localhost:8025`

État de l'observabilité : le programme couvre exactement les deux volets exigés par le référentiel (cités en introduction de ce chapitre) – un tableau de bord d'indicateurs de service (Grafana, BI) et un monitorage avec alerte sur rupture de service (schéma `monitoring`, règle Grafana → webhook → e-mail) – plus une journalisation des opérations (schéma `gouvernance`). Ce que le programme **n'a pas** : une agrégation centralisée des logs applicatifs entre conteneurs (pas de Loki/ELK, seul `docker compose logs` par service) ni de traçage distribué (*tracing*) entre les appels API/ETL. Ni l'un ni l'autre n'est exigé par les critères du référentiel cités plus haut, et leur ajout serait disproportionné à l'échelle du jeu de données du programme – même principe de sobriété que le choix, déjà justifié ailleurs (chapitre 3 §C9), de ne pas déployer d'outil « big data » séparé pour 25 000 lignes.

6.5 Rejouer le remplissage des données et les pipelines

Une fois les services lancés (section précédente), les trois briques de stockage se peuplent par une séquence de commandes Python (`python3 -m datacore.<module>`), jamais par insertion manuelle – rejouable à l'identique en démonstration, y compris à l'oral.

Base de staging (C8-C11, chapitre 3) :

Listing 6.2 – Extraction, nettoyage puis chargement de la base de staging

```
python3 -m datacore.ingestion.run_extraction    # C8 : 5 sources -> data/interim/
python3 -m datacore.processing.run_cleaning     # C10 : dedoublonnage fichiers clients
alembic upgrade head                            # C11 : schema MERISE
python3 -m datacore.storage.staging.load_staging
```

Entrepôt OMEGA BI (C13-C17, chapitre 4) :

Listing 6.3 – Création puis chargement ETL de l'entrepôt

```
./scripts/init_omega_bi_db.sh                          # C14 : base + role bi_reader
alembic -c alembic_omega_bi.ini upgrade head           # C14 : 3 schemas + 9 tables
python3 -m datacore.storage.warehouse.load_dim_temps   # dimension calendaire generee
python3 -m datacore.storage.warehouse.load_warehouse   # C15 : ETL staging -> entrepot
```

Data lake OMEGA LAKE (C18-C21, chapitre 5) :

Listing 6.4 – Ingestion puis transformation du data lake

```
python3 -m datacore.storage.lake.ingestion_batch   # C19 : 4 flux IoT batch -> raw/
python3 -m datacore.storage.lake.sse_consumer      # C19 : flux temps reel -> raw/ (continu)
python3 -m datacore.storage.lake.transform         # C18 : raw/ -> staging/ -> curated/
python3 -m datacore.storage.lake.catalogue         # C20 : catalogue par introspection
python3 -m datacore.storage.lake.curated_bi        # C21 : export pseudonymise
```

6.5.1 Transactions et rollback

Les trois chargements (`load_staging`, `load_warehouse`) s'exécutent dans une **transaction unique** : un `COMMIT` unique après la dernière étape réussie, un `ROLLBACK` complet sur toute exception – jamais de base partiellement peuplée :

Listing 6.5 – Transaction unique du pipeline ETL entrepôt (`src/datacore/storage/warehouse/load_warehouse.py`)

```
try:
    truncate_warehouse(warehouse_conn)
    # ... chargement des 6 dimensions puis des 3 faits ...
    warehouse_conn.commit()
except Exception:
    warehouse_conn.rollback()
    raise
finally:
    staging_conn.close()
    warehouse_conn.close()
```

Point plus subtil : l'exécution est journalisée (`gouvernance.journal_operations`, C16) sur une **connexion Postgres séparée** de celle du chargement, committée immédiatement à l'ouverture (statut `en_cours`) puis mise à jour à la sortie du bloc. Si le chargement échoue et annule sa propre transaction, la ligne de journal indiquant l'échec, elle, survit – un `ROLLBACK` ne peut pas invalider un `COMMIT` déjà passé sur une autre connexion. La contrainte d'unicité SCD2 (`uq_dim_client_courant`, chapitre 4 §C17) est elle aussi traduite en diagnostic explicite plutôt que laissée remonter telle quelle :

Listing 6.6 – Traduction d'une violation de contrainte en diagnostic (`src/datacore/storage/warehouse/load_warehouse.py`)

```
except psycopg2.errors.UniqueViolation as exc:
    raise RuntimeError(
        f"Conflit SCD2 sur dim_client : une version courante existe "
        f"deja pour client_id={r['id']} (contrainte "
        f"uq_dim_client_courant) -- execution concurrente du "
        f"pipeline ou ecriture directe en base ?"
    ) from exc
```

6.5.2 Visualiser les pipelines

Aucun orchestrateur à interface graphique (pas d'Airflow, pas de Dagster) : chaque pipeline est un script Python invoqué directement, choix de sobriété cohérent avec l'échelle du programme

(même principe que pour C9 ci-dessus). Trois façons concrètes de « voir » un pipeline malgré tout :

- **La topologie**, statique : la vue d'ensemble staging/entrepôt/lake (figure 5.1, chapitre 5 §C18) montre l'enchaînement réel des étapes, texte Mermaid versionné dans le dépôt.
- **Le résultat d'une exécution**, après coup : chaque script affiche un résumé en sortie (lignes chargées, doublons résolus), et pour l'entrepôt, `gouvernance.journal_operations` garde un historique interrogeable (début, fin, statut, détail) – visible dans Grafana via `gouvernance.v_dernieres_operations`.
- **La fraîcheur des données du lake**, en continu : le dashboard « OMEGA LAKE — Monitoring » (C20bis, section précédente) trace la dernière ingestion par flux.

Aucun outil ne montre un graphe d'exécution animé en temps réel (type DAG Airflow) – absent du référentiel de compétences et disproportionné pour des pipelines de quelques minutes exécutés à la demande, pas en continu.

Pourquoi pas un orchestrateur (Airflow, Dagster) : question posée explicitement au formateur plutôt que tranchée seule – réponse confirmée, un orchestrateur n'est pas exigé pour valider la compétence, mais constitue un plus. Le choix de ne pas en intégrer un à ce stade reste délibéré, pour trois raisons : le référentiel (C8, C15, C19) n'exige explicitement aucun outil d'orchestration précis, le coût d'intégration est concret et non négligeable (un orchestrateur ajoute au minimum trois services supplémentaires – initialisation, ordonnanceur, interface web – plus sa propre base de métadonnées, sur une infrastructure qui en compte déjà huit), et surtout aucun pipeline du programme ne s'exécute aujourd'hui selon une cadence récurrente réelle à orchestrer – chacun est invoqué à la demande sur un jeu de données statique. Un orchestrateur ajouté maintenant répondrait à un besoin plus démonstratif qu'opérationnel.

6.6 Portée budgétaire

Comme les jalons transverses de rédaction du rapport et de préparation de la soutenance, M4 n'a jamais eu de ligne dédiée dans le budget prévisionnel initial (180 000 €, voir chapitre 2) – staffing interne uniquement, les outils retenus (Grafana, MailHog) étant open source et auto-hébergés. Aucun dépassement constaté.

7. Synthèse transverse

8. Conclusion

A. Glossaire métier

Glossaire complet dans `docs/architecture/topographie_donnees.md` §1 (C2) ; repris ici in extenso pour que le rapport reste autosuffisant. Chaque terme est une cible de renvoi (voir par exemple SCD2 au chapitre 4) – cliquable dans le PDF.

Terme	Définition
3PL (*Third-Party Logistics*)	Prestataire logistique externalisé gérant entreposage, préparation de commandes et transport pour le compte de tiers (les clients d'Omega Logistics).
WMS (*Warehouse Management System*)	Système de gestion d'entrepôt. Chez Omega Logistics : **FluxPro**, qui pilote commandes, stocks, produits et expéditions.
TMS (*Transport Management System*)	Système de gestion du transport. Chez Omega Logistics : **TransFlow**, qui organise tournées et livraisons de la flotte.
Entrepôt	Site d'exploitation d'Omega Logistics. Trois sites : Lyon (`OMG-LYO`), Lille (`OMG-LIL`), Marseille (`OMG-MAR`).
Client	Donneur d'ordre d'Omega Logistics. Trois clients fictifs : NordDrive (pièces auto), FreshMarket (grande distribution alimentaire), MedioTex (textile).
Commande	Demande d'expédition de produits passée par un client pour un entrepôt donné, composée d'une ou plusieurs lignes de commande.
Expédition	Envoi physique résultant d'une commande, confié à un transporteur, identifié par un numéro de suivi (`tracking_number`).
Stock	Quantité disponible d'un produit dans un entrepôt à une date de mise à jour donnée.
SKU (*Stock Keeping Unit*)	Référence unique d'un produit dans le référentiel FluxPro.
Tournée	Ensemble de livraisons affectées à un chauffeur et un véhicule, pour une date donnée, organisé par TransFlow.
Livraison	Remise d'un colis (une expédition) à une adresse, rattachée à une tournée, avec un statut et des horaires estimé/réel.
Transporteur	Société assurant le transport physique des expéditions (ex. RapidFret). Distinct d'Omega Logistics, dont TransFlow orchestre les tournées.
SLA (*Service Level Agreement*)	Accord de niveau de service définissant la qualité de prestation attendue.

Terme	Définition
RFID (*Radio Frequency Identification*)	Technologie d'étiquetage/scan sans contact utilisée pour tracer les palettes en entrepôt.
IoT (*Internet of Things*)	Capteurs connectés en entrepôt (température, comptage caméra) et sur la flotte (géolocalisation) – bloc 4 (data lake).
SSE (*Server-Sent Events*)	Protocole HTTP de flux continu à sens unique serveur → client (une connexion ouverte, le serveur pousse des évènements). Utilisé par `/api/stream/capteurs` pour diffuser les mesures capteurs en temps réel (bloc 4).
Staging	Base de données de travail intermédiaire, cible de la collecte du bloc 2, alimentant ensuite l'entrepôt de données (bloc 3). À ne pas confondre avec la zone `staging` du data lake (bloc 4), qui porte le même nom par convention Kimball/data lake mais désigne une brique de stockage différente.
SCD1 (*Slowly Changing Dimension*, type 1)	Historisation Kimball où un changement **écrase** la valeur précédente, sans garder de trace. Cas implicite dans OMEGA BI : `Dim_Client.code` n'est jamais versionné.
SCD2 (*Slowly Changing Dimension*, type 2)	Historisation Kimball où un changement produit une **nouvelle ligne** (nouvelle clé de substitution, `valid_from`/ `valid_to`/`is_current`), en conservant l'ancienne. Implémentée sur `Dim_Client.nom`/`secteur` en C17 – voir chapitre 4.
S3 (*Simple Storage Service*)	Service de stockage objet d'AWS ; son API HTTP est devenue un standard de fait, repris par des solutions non-AWS comme **MinIO** (bloc 4) pour rester compatibles avec les mêmes outils sans dépendre d'AWS.
Bucket	Conteneur de premier niveau d'un stockage objet S3/MinIO (grossièrement, un répertoire racine). OMEGA LAKE utilise un **bucket unique**, `omega-lake` – les zones `raw`/`staging`/`curated` sont des préfixes à l'intérieur de ce bucket, pas des buckets séparés.
Zone (data lake)	Une étape de transformation à l'intérieur du bucket unique (`raw`/`staging`/`curated`) – un préfixe de chemin, pas un bucket séparé.
Flux (data lake)	Une source de données IoT distincte, avec son propre schéma (ex. `capteurs_temperature`, `rfid_scans`) – un sous-préfixe à l'intérieur de chaque zone. 5 flux au total.
httpfs	Extension DuckDB qui sait lire/écrire des fichiers via HTTP(S) ou `s3://`, y compris contre un stockage S3-compatible non-AWS comme MinIO – nécessite alors une configuration explicite d'endpoint (`s3_endpoint`) et de style d'adressage (`s3_url_style='path'`), MinIO n'étant pas AWS S3.
CoSu (Comité de suivi)	Réunion mensuelle de pilotage entre le chef·fe de projet, le commanditaire opérationnel (Karim BELAÏD) et la sponsor (Éléonore RAKOTO).
Diagramme de Gantt	Représentation graphique d'un planning : une barre horizontale par tâche, positionnée dans le temps, montrant enchaînements et chevauchements entre phases – voir figure 2.2 [1].

1. Maintenu en Mermaid (texte versionné dans le dépôt) plutôt que dans un outil graphique externe.

Terme	Définition
BI (*Business Intelligence*)	Ensemble des pratiques et outils visant l'**aide à la prise de décision** à partir de données consolidées (tableaux de bord, indicateurs, restitution visuelle) – par opposition à la donnée brute ou à son seul stockage. Chez Omega Logistics : l'entrepôt **OMEGA BI** (bloc 3) et sa restitution dans **Grafana** (C16bis).
RGPD (Règlement Général sur la Protection des Données)	Règlement européen encadrant le traitement des données personnelles. Mis en œuvre via un registre des traitements tenu à chaque brique (staging, entrepôt, lake) et la pseudonymisation des identifiants sensibles.
RGAA (Référentiel Général d'Amélioration de l'Accessibilité)	Référentiel français d'accessibilité numérique (contrastes, navigation clavier, alternatives textuelles), appliqué à l'ensemble des livrables du programme.
RGESN (Référentiel Général d'Écoconception de Services Numériques)	Référentiel français de sobriété numérique, mobilisé dans les choix d'architecture (open source, auto-hébergé, dimensionnement au volume réel plutôt que par principe).
RICE (*Reach, Impact, Confidence, Effort*)	Méthode de priorisation de chantiers : score = (Reach × Impact × Confidence) / Effort. Utilisée pour arbitrer le séquencement du programme en C1.
SMART (Spécifique, Mesurable, Acceptable/Ambitieux, Réaliste, Temporellement défini)	Grille de formulation d'objectifs de projet. Les 5 objectifs du programme DATA CORE sont formulés selon cette grille.
ETL (*Extract, Transform, Load*)	Chaîne de traitement qui extrait les données d'une source, les transforme (nettoyage, conformité au schéma cible) puis les charge dans leur destination. Chez Omega Logistics : les pipelines qui peuplent l'entrepôt OMEGA BI depuis la base de staging (C15).
Datamart	Sous-ensemble de l'entrepôt de données construit autour d'un processus métier précis, plutôt qu'un modèle unique monolithique. OMEGA BI est construit **par datamart** (« Exploitation » et « Commercial »), partageant des dimensions conformées.

B. Matrice compétences